

CS224N | Assignment 2

Selene · August 25, 2026

1 Problem 1

(a)

Let $y \in \mathbb{R}^{|V|}$ be the one-hot true distribution, let $\hat{y} \in \mathbb{R}^{|V|}$ be the predicted distribution, and let

$$\hat{y}_w = P(O = w \mid C = c) = \frac{\exp(u_w^\top v_c)}{\sum_{j \in V} \exp(u_j^\top v_c)}.$$

The matrix U contains the outside-word vectors as its columns:

$$U = [u_1 \quad u_2 \quad \cdots \quad u_{|V|}].$$

Since y is one-hot, $y_o = 1$ and $y_w = 0$ for every $w \neq o$. Therefore,

$$-\sum_{w \in V} y_w \log \hat{y}_w = -\log \hat{y}_o.$$

(b)

(i)

First, rewrite the naive-softmax loss as

$$J_{\text{naive-softmax}}(v_c, o, U) = -\log \hat{y}_o = -u_o^\top v_c + \log \left(\sum_{w \in V} \exp(u_w^\top v_c) \right).$$

Taking the derivative with respect to v_c ,

$$\frac{\partial J}{\partial v_c} = -u_o + \frac{\sum_{w \in V} \exp(u_w^\top v_c) u_w}{\sum_{j \in V} \exp(u_j^\top v_c)} = -u_o + \sum_{w \in V} \hat{y}_w u_w.$$

Because y is one-hot,

$$u_o = Uy, \quad \sum_{w \in V} \hat{y}_w u_w = U\hat{y}.$$

Hence the vectorized result is

$$\frac{\partial J}{\partial v_c} = U(\hat{y} - y)$$

(ii)

The gradient is zero exactly when

$$U(\hat{y} - y) = 0$$

Equivalently, $\hat{y} - y \in \text{Null}(U)$. In particular, $\hat{y} = y$ is sufficient for the gradient to be zero.

(iii)

Using a learning rate $\eta > 0$, gradient descent gives

$$v_c \leftarrow v_c - \eta(U\hat{y} - Uy) = v_c + \eta u_o - \eta \sum_{w \in V} \hat{y}_w u_w.$$

The $+\eta u_o$ term moves the center-word vector v_c toward the true outside-word vector u_o , increasing their similarity. The $-\eta \sum_w \hat{y}_w u_w$ term moves v_c away from the probability-weighted average of the predicted outside-word vectors, with a larger correction for words to which the model currently assigns more probability. Together, these changes increase the score of the true outside word relative to the scores of the other words.

(c)

Suppose $u_x = \alpha u_y$ for $x \neq y$. If $\alpha > 0$, then

$$\frac{u_x}{\|u_x\|_2} = \frac{\alpha u_y}{\|\alpha u_y\|_2} = \frac{u_y}{\|u_y\|_2}.$$

Thus, L2 normalization makes two vectors that point in the same direction identical, even if their original magnitudes differ. It removes useful information when vector magnitude encodes information relevant to the phrase classification—for example, when x and y have similar semantic directions but different strengths or polarities in the classifier. It does not remove useful information when only vector direction matters, or when same-direction vectors with different magnitudes should have the same effect on the final classification.

(d)

Since U is formed by placing the outside-word vectors in columns, its gradient has the same column structure:

$$\frac{\partial J(v_c, o, U)}{\partial U} = \begin{bmatrix} \frac{\partial J(v_c, o, U)}{\partial u_1} & \frac{\partial J(v_c, o, U)}{\partial u_2} & \dots & \frac{\partial J(v_c, o, U)}{\partial u_{|V|}} \end{bmatrix}$$

(e)

The loss is

$$J = -u_o^\top v_c + \log \left(\sum_{j \in V} \exp(u_j^\top v_c) \right).$$

For the true outside word $w = o$,

$$\frac{\partial J}{\partial u_o} = -v_c + \frac{\exp(u_o^\top v_c)}{\sum_{j \in V} \exp(u_j^\top v_c)} v_c = (\hat{y}_o - 1)v_c.$$

For every $w \neq o$,

$$\frac{\partial J}{\partial u_w} = \frac{\exp(u_w^\top v_c)}{\sum_{j \in V} \exp(u_j^\top v_c)} v_c = \hat{y}_w v_c.$$

Therefore,

$$\frac{\partial J}{\partial u_w} = (\hat{y}_w - y_w)v_c = \begin{cases} (\hat{y}_o - 1)v_c, & w = o, \\ \hat{y}_w v_c, & w \neq o. \end{cases}$$

(f)

For $f(x) = \max(\alpha x, x)$, where $0 < \alpha < 1$,

$$f'(x) = \begin{cases} 1, & x > 0, \\ \alpha, & x < 0. \end{cases}$$

The derivative at $x = 0$ is not required.

(g)

Starting from

$$\sigma(x) = \frac{1}{1 + e^{-x}},$$

we obtain

$$\sigma'(x) = \frac{e^{-x}}{(1 + e^{-x})^2} = \frac{1}{1 + e^{-x}} \left(1 - \frac{1}{1 + e^{-x}} \right).$$

Hence,

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

2 Problem 2

(a)

(i)

Momentum averages gradients over multiple minibatches, so random variations in individual minibatch gradients tend to cancel out. Directions that are consistently useful accumulate in m , making parameter updates smoother and less noisy. This can reduce oscillation and allow the model to converge more stably and quickly.

(ii)

Parameters whose gradients have historically been small receive relatively larger updates, since their corresponding values in v are smaller. Parameters with consistently large gradients receive smaller updates. This adaptive scaling prevents large-gradient parameters from changing too aggressively while still allowing small-gradient parameters to make meaningful progress.

(b)

(i)

For each hidden unit i ,

$$(h_{\text{drop}})_i = \gamma d_i h_i.$$

Since $\mathbb{E}[d_i] = 1 - p_{\text{drop}}$,

$$\mathbb{E}[(h_{\text{drop}})_i] = \gamma h_i (1 - p_{\text{drop}}).$$

Requiring $\mathbb{E}[(h_{\text{drop}})_i] = h_i$ gives

$$\gamma = \frac{1}{1 - p_{\text{drop}}}.$$

(ii)

Dropout is applied during training as a regularization method, since randomly dropping hidden units prevents the model from relying too heavily on specific neurons and reduces overfitting. During evaluation, dropout is disabled so that the full network is used and predictions remain stable and deterministic.

3 Problem 3

(a)

The required sequence of transitions is:

Stack	Buffer	New dependency	Transition
[ROOT]	[I, presented, my, findings, at, the, NLP, conference]		Initial Configuration
[ROOT, I]	[presented, my, findings, at, the, NLP, conference]		SHIFT
[ROOT, I, presented]	[my, findings, at, the, NLP, conference]		SHIFT
[ROOT, presented]	[my, findings, at, the, NLP, conference]	presented $\rightarrow$ I	LEFT-ARC
[ROOT, presented, my]	[findings, at, the, NLP, conference]		SHIFT
[ROOT, presented, my, findings]	[at, the, NLP, conference]		SHIFT
[ROOT, presented, findings]	[at, the, NLP, conference]	findings $\rightarrow$ my	LEFT-ARC
[ROOT, presented]	[at, the, NLP, conference]	presented $\rightarrow$ findings	RIGHT-ARC
[ROOT, presented, at]	[the, NLP, conference]		SHIFT
[ROOT, presented, at, the]	[NLP, conference]		SHIFT

Stack	Buffer	New dependency	Transition
[ROOT, presented, at, the, NLP]	[conference]		SHIFT
[ROOT, presented, at, the, NLP, conference]	[]		SHIFT
[ROOT, presented, at, the, conference]	[]	conference → NLP	LEFT-ARC
[ROOT, presented, at, conference]	[]	conference → the	LEFT-ARC
[ROOT, presented, conference]	[]	conference → at	LEFT-ARC
[ROOT, presented]	[]	presented → conference	RIGHT-ARC
[ROOT]	[]	ROOT → presented	RIGHT-ARC

(b)

A sentence containing n words requires $2n$ transitions to parse. Each word is shifted onto the stack exactly once and is removed from the stack exactly once by either a LEFT-ARC or RIGHT-ARC transition.

(c)

```

1 class PartialParse(object):
2     def __init__(self, sentence):
3         """Initializes this partial parse.
4
5         @param sentence (list of str): The sentence to be parsed as a list of words.
6             Your code should not modify the sentence.
7
8         """
9         # The sentence being parsed is kept for bookkeeping purposes. Do NOT alter it in your code.
10        self.sentence = sentence
11        self.stack = ['ROOT']
12        self.buffer = list(sentence)
13        self.dependencies = []

```

Figure 1: Parser implementation (first excerpt).

```

1     def parse_step(self, transition):
2         """Performs a single parse step by applying the given transition to this partial parse
3
4         @param transition (str): A string that equals "S", "LA", or "RA" representing the shift,
5             left-arc, and right-arc transitions. You can assume the provided
6             transition is a legal transition.
7
8         """
9         ### YOUR CODE HERE (~7-12 Lines)
10        if transition == "S":
11            self.stack.append(self.buffer.pop(0))
12
13        elif transition == "LA":
14            head = self.stack[-1]
15            dependent = self.stack.pop(-2)
16            self.dependencies.append((head, dependent))
17
18        elif transition == "RA":
19            dependent = self.stack.pop()
20            head = self.stack[-1]
21            self.dependencies.append((head, dependent))

```

Figure 2: PartialParse transition implementation.

(d)

`minibatch_parse` creates one `PartialParse` object for each input sentence and keeps their original order. During each iteration, it selects up to `batch_size` unfinished parses, asks the model to predict one transition for each parse, and applies those transitions. A parse is removed from the unfinished list only when its buffer is empty and its stack contains only `ROOT`. Finally, dependencies are collected from the original list of partial parses, which preserves the input sentence order.

```

1 def minibatch_parse(sentences, model, batch_size):
2     """Parses a list of sentences in minibatches using a model.
3
4     @param sentences (list of list of str): A list of sentences to be parsed
5         (each sentence is a list of words and each word is of type string)
6     @param model (ParserModel): The model that makes parsing decisions. It is assumed to have a function
7         model.predict(partial_parses) that takes in a list of PartialParses as input and
8         returns a list of transitions predicted for each parse. That is, after calling
9         transitions = model.predict(partial_parses)
10        transitions[i] will be the next transition to apply to partial_parses[i].
11     @param batch_size (int): The number of PartialParses to include in each minibatch
12
13
14     @return dependencies (list of dependency lists): A list where each element is the dependencies
15         list for a parsed sentence. Ordering should be the
16         same as in sentences (i.e., dependencies[i] should
17         contain the parse for sentences[i]).
18     """
19     dependencies = []
20
21     ### YOUR CODE HERE (~8-10 Lines)
22     partial_parses = [PartialParse(sentence) for sentence in sentences]
23     unfinished_parses = partial_parses[:]
24
25     while unfinished_parses:
26         minibatch = unfinished_parses[:batch_size]
27         transitions = model.predict(minibatch)
28         for partial_parse, transition in zip(minibatch, transitions):
29             partial_parse.parse_step(transition)
30             unfinished_parses = [
31                 partial_parse for partial_parse in unfinished_parses
32                 if partial_parse.buffer or len(partial_parse.stack) > 1
33             ]
34
35     dependencies = [partial_parse.dependencies for partial_parse in partial_parses]
36
37     return dependencies

```

(e)

(i)

Let

$$\mathbf{z} = \mathbf{x}\mathbf{W} + \mathbf{b}_1, \quad \mathbf{h} = \text{ReLU}(\mathbf{z}).$$

For an individual hidden unit h_i ,

$$h_i = \max(z_i, 0) = \max\left(\sum_k x_k W_{ki} + b_{1,i}, 0\right).$$

Therefore, for an input feature x_j ,

$$\begin{cases} W_{ji}, & z_i > 0, \\ 0, & z_i < 0. \end{cases}$$

The derivative is undefined at $z_i = 0$, which the question allows us to ignore.

(ii)

Let the logits be $\mathbf{l}$, and let

$$\hat{y}_j = \frac{e^{l_j}}{\sum_k e^{l_k}}$$

be the softmax probability for class j . The cross-entropy loss is

$$-\sum_j y_j \log \hat{y}_j.$$

For a one-hot target vector whose correct class is c , the derivative with respect to logit l_i is

$$\hat{y}_i - y_i.$$

Equivalently,

$$\begin{cases} \hat{y}_i - 1, & i = c, \\ \hat{y}_i, & i \neq c. \end{cases}$$

This gradient increases the logit of the correct transition and decreases the relative preference for incorrect transitions during training.

(iii) UAS

```
Epoch 8 out of 10
100% | 1848/1848 [00:39<00:00, 47.20it/s]
Average Train Loss: 0.07237303829489848
Evaluating on dev set
1445850it [00:00, 104824974.74it/s]
- dev UAS: 88.59
New best dev UAS! Saving model.

Epoch 9 out of 10
100% | 1848/1848 [00:39<00:00, 47.20it/s]
Average Train Loss: 0.06922483758686425
Evaluating on dev set
1445850it [00:00, 105115690.88it/s]
- dev UAS: 88.85
New best dev UAS! Saving model.

Epoch 10 out of 10
100% | 1848/1848 [00:39<00:00, 47.16it/s]
Average Train Loss: 0.06653498037561
Evaluating on dev set
1445850it [00:00, 104870293.09it/s]
- dev UAS: 88.72
```

```
=====
TESTING
=====

Restoring the best model weights found on the dev set
Final evaluation on test set
2919736it [00:00, 152889054.59it/s]
- test UAS: 89.23
Done!
```

The best development-set UAS achieved by the model was **88.85**. The final model, restored using the best development-set checkpoint, achieved a test-set UAS of **89.23**.

UAS measures the percentage of words whose predicted syntactic heads are correct, regardless of dependency-relation labels.

My code

```

1  def __init__(self, embeddings, n_features=36,
2      hidden_size=200, n_classes=3, dropout_prob=0.5):
3      """ Initialize the parser model.
4
5      @param embeddings (ndarray): word embeddings (num_words, embedding_size)
6      @param n_features (int): number of input features
7      @param hidden_size (int): number of hidden units
8      @param n_classes (int): number of output classes
9      @param dropout_prob (float): dropout probability
10     """
11     super(ParserModel, self).__init__()
12     self.n_features = n_features
13     self.n_classes = n_classes
14     self.dropout_prob = dropout_prob
15     self.embed_size = embeddings.shape[1]
16     self.hidden_size = hidden_size
17     self.embeddings = nn.Parameter(torch.tensor(embeddings))
18
19     ### YOUR CODE HERE (~9-10 Lines)
20     self.embed_to_hidden_weight = nn.Parameter(
21         torch.empty(self.n_features * self.embed_size, self.hidden_size))
22     self.embed_to_hidden_bias = nn.Parameter(torch.empty(self.hidden_size))
23     nn.init.xavier_uniform_(self.embed_to_hidden_weight)
24     nn.init.uniform_(self.embed_to_hidden_bias)
25
26     self.dropout = nn.Dropout(self.dropout_prob)
27
28     self.hidden_to_logits_weight = nn.Parameter(
29         torch.empty(self.hidden_size, self.n_classes))
30     self.hidden_to_logits_bias = nn.Parameter(torch.empty(self.n_classes))
31     nn.init.xavier_uniform_(self.hidden_to_logits_weight)
32     nn.init.uniform_(self.hidden_to_logits_bias)

```

Figure 3: Parser model initialization.

```

1  def embedding_lookup(self, w):
2      """ Utilize `w` to select embeddings from embedding matrix `self.embeddings`
3      @param w (Tensor): input tensor of word indices (batch_size, n_features)
4
5      @return x (Tensor): tensor of embeddings for words represented in w
6                          (batch_size, n_features * embed_size)
7      """
8
9      ### YOUR CODE HERE (~1-4 Lines)
10     x = self.embeddings[w].view(w.shape[0], -1)
11     return x

```

```

1 def forward(self, w):
2     """ Run the model forward.
3
4     Note that we will not apply the softmax function here because it is included in the loss function nn.CrossEntropyLoss
5
6     PyTorch Notes:
7     - Every nn.Module object (PyTorch model) has a 'forward' function.
8     - When you apply your nn.Module to an input tensor 'w' this function is applied to the tensor.
9       For example, if you created an instance of your ParserModel and applied it to some 'w' as follows,
10      the 'forward' function would be called on 'w' and the result would be stored in the 'output' variable:
11          model = ParserModel()
12          output = model(w) # this calls the forward function
13      - For more details checkout: https://pytorch.org/docs/stable/nn.html#torch.nn.Module.forward
14
15     @param w (Tensor): input tensor of tokens (batch_size, n_features)
16
17     @return logits (Tensor): tensor of predictions (output after applying the layers of the network)
18                             without applying softmax (batch_size, n_classes)
19     """
20     ### YOUR CODE HERE (~3-5 lines)
21     x = self.embedding_lookup(w)
22     hidden = F.relu(torch.matmul(x, self.embed_to_hidden_weight) + self.embed_to_hidden_bias)
23     hidden = self.dropout(hidden)
24     logits = torch.matmul(hidden, self.hidden_to_logits_weight) + self.hidden_to_logits_bias
25

```

Test

```

(cs224n_a2) ✖ cs224n_a2 ~/Projects/cs224n } master ± cd assignment2
python parser_transitions.py part_c
SHIFT test passed!
LEFT-ARC test passed!
RIGHT-ARC test passed!
parse test passed!
(cs224n_a2) 🐞 cs224n_a2 ~/Projects/cs224n/assignment2 } master ± cd ~/Projects/cs224n/assignment2
python parser_transitions.py part_d
minibatch_parse test passed!
(cs224n_a2) 🐞 cs224n_a2 ~/Projects/cs224n/assignment2 } master ± cd ~/Projects/cs224n/assignment2
python parser_model.py -e -f
Embedding_lookup sanity check passes!
Forward sanity check passes!
(cs224n_a2) 🐞 cs224n_a2 ~/Projects/cs224n/assignment2 } master ± |

```